

TEAM SHARING

AI 时代的工程师

颠覆、困境与进化

Guohua Liang · 2026.07

当 AI 开始写代码.....

70%

GitHub Copilot 用户表示
编码效率显著提升

10x

AI 辅助下代码生成
速度对比手写

40%

AI 生成的代码已占
新代码库比例 (2026)

AI 不是在取代程序员，而是在重新定义“程序”本身。
当写代码的门槛趋近于零，工程师的核心价值在哪里？

“日抛程序”时代

代码正在变成快消品

什么是“日抛程序”？

- AI 一分钟生成的脚本，用完即弃
- 不需维护、不需测试、不需文档
- 解决一次性问题的“即用即扔”代码
- 从“精心雕琢”到“快速消费”

这意味着什么？

- 代码单次价值↓，解题能力价值↑
- 写代码≠软件工程，核心是可维护性
- “日抛”适合探索原型，不适合生产
- 真正挑战：从“能用”到“能持续用”

但现实不只有新代码

谁在维护那 80% 的存量系统？



遗留系统

企业核心业务跑在 10+ 年前的代码上。COBOL、Java 6、自研框架——AI 看不懂上下文，改不动。



领域知识

业务规则藏在代码和工程师脑子里，不是 prompt 能提取的。AI 写得出语法，写不出“为什么这样写”。



系统集成

分布式系统的一致性、容灾、灰度发布——需要对整体架构的深刻理解，不是单个函数能解决的。

历史的镜子

马车 → 汽车：那些赶马车的人后来怎么样了？



关键洞察：

新技术从来不是一夜之间取代旧技术。过渡期往往长达 20-30 年，真正消失的不是人，是那些拒绝学习新工具的人。

每次技术革命的共同规律

历史总是惊人地相似

打字机 → 文字处理器

消失的是“打字”岗位，留下的是“文档处理”能力

手工制图 → CAD

工具放大了能力，但没有取消能力本身

人工交易 → 算法交易

顶级交易员转型为策略设计者，理解市场的人不可替代

胶片 → 数码摄影

暗房技术消失，但构图、光线、叙事能力更被重视

技术替代的三个阶段

我们现在处于哪里？

01

恐慌期

“AI 要取代我了！”
媒体渲染、焦虑蔓延
过度悲观的预测

02

共存期

人 + AI 协作模式成型
部分重复工作被取代
新的工作方式涌现

03

融合期

AI 成为基础设施
工程师身份重新定义
新岗位和价值体系形成

↑ 我们在这里

能力价值迁移图谱

↓ 正在贬值

- 纯手写 CRUD 代码的能力
- 记忆 API / 语法细节
- 按规范搬砖的执行力
- 单一语言 / 框架的熟练度
- 重复性测试用例编写

↑ 正在增值

- 系统设计 & 架构决策能力
- 问题定义 & 需求拆解能力
- 跨领域知识 & 业务理解
- AI 协作 & Prompt 工程能力
- 代码审查 & 质量判断能力

工程师的三种进化方向

架构师路线

从写代码到设计系统

- 掌控复杂系统全局
- 技术选型 & 权衡取舍
- AI 无法替代的全局视野

产品工程师路线

从实现需求到定义问题

- 理解业务, 用技术解决真实问题
- 需求拆解 & 优先级判断
- 成为业务和技术的桥梁

AI 原生工程师

从写代码到指挥 AI 写代码

- Prompt 工程 & AI 工作流
- AI Agent 架构与编排
- 人机协作模式的先行者

核心身份转型

过去:写代码的人

- 价值 = 代码行数 × 熟练度
- 深耕一门语言 / 框架
- 需求给什么就做什么
- 衡量标准:写得快、bug 少
- 工具:IDE + Stack Overflow



未来:解决问题的人

- 价值 = 问题定义 × 方案选择
- 理解多领域, 善于跨界整合
- 主动发现和定义问题
- 衡量标准:解决得好、影响力大
- 工具:AI Agent + 系统思维

给工程师的行动建议

现在就可以开始做的 5 件事

1

拥抱 AI 工具

把 Copilot / Cursor 用起来, 先做增量, 感受AI 的能力边界

2

练习定义问题

拿到需求先问“为什么要做”, 而不是“怎么实现”

3

建立系统思维

读架构书、画系统图、参与技术评审, 从函数级升维到系统级

4

深耕业务领域

成为某个领域的专家。AI 最难替代“懂这个行业”的人

5

保持学习习惯

不是学新语言, 而是学新思维, 关注 AI 原生的工作方式

马车夫的故事告诉我们：

淘汰你的不是新技术，
而是你对新技术的态度。

这不是终点，是新的起点。
与 AI 同行，做不可替代的工程师。